



INSTITUT
POLYTECHNIQUE
DE PARIS

Documentation Technique

Tetris Multiplayer

IN204 : Programmation Objet et Génie Logiciel

Alfredo Quintella Pinto

Helena Guachalla de Andrade

Table des matières

1	Introduction	2
2	Fonctionnalités	2
2.1	Description des Fonctions	2
2.2	Jouabilité et Contrôles	3
2.2.1	Mode Un Joueur	3
2.2.2	Mode Deux Joueurs Local	3
3	Architecture & Implementation	4
3.1	Machine à États Finis	5
3.2	Composants MVC	5
3.2.1	Modèle (GameData, ClassicBoard, Tetromino)	5
3.2.2	Vue (GameRenderer, BoardRenderer, LayoutManager)	7
3.2.3	Contrôleur (GameController)	7
3.3	Réseau (NetworkManager)	8
3.4	Boucle de Jeu Principale	9
3.5	Utils	9
4	Conclusion	9

1 Introduction

Ce document présente le rapport technique du projet de développement d'un jeu Tetris classique, avec une extension multijoueur en réseau. L'objectif principal est de concevoir une application robuste et performante, alliant les mécaniques de jeu traditionnelles de Tetris à des fonctionnalités de compétition en temps réel entre plusieurs joueurs.

Pour atteindre cet objectif, ce projet s'appuie sur la bibliothèque C++ SFML (*Simple and Fast Multimedia Library*) qui fournit tous les composants nécessaires au développement. SFML est utilisée pour la gestion de l'interface graphique, du rendu visuel, des entrées du joueur, ainsi que pour les communications réseau via son module de sockets TCP/IP.

Ce rapport détaillera l'architecture générale et les fonctionnalités implémentées pour offrir une expérience de jeu engageante.

2 Fonctionnalités

2.1 Description des Fonctions

Les principales fonctions implémentées dans le jeu Tetris sont les suivantes :

Génération et manipulation des pièces :

- Création aléatoire des 7 types de pièces (I, O, T, L, J, S, Z) avec leurs formes et couleurs spécifiques
- Rotation de 90° dans le sens horaire
- Déplacement latéral (gauche/droite)
- Système de *ghost piece* qui projette la position finale de la chute
- Fonction *hold* pour conserver une pièce en réserve
- *Wall kick* pour les rotations près des bordures

Gestion de la chute :

- Descente automatique à vitesse variable selon le niveau
- Accélération manuelle par l'utilisateur (*soft drop*)
- Chute instantanée (*hard drop*)

Détection de lignes complètes :

- Vérification des lignes entièrement remplies
- Effacement des lignes complètes
- Tassement des lignes supérieures

Détection de fin de jeu : Arrêt du jeu lorsqu'une nouvelle pièce ne peut pas être placée (première ligne occupée).

Communication réseau :

- Système client/serveur utilisant TCP/IP
- Synchronisation des états de jeu entre deux joueurs
- Gestion de lobby pour connexion

Gestion des menus :

- Menu principal avec navigation
- Écran de fin de jeu avec score final
- Options de redémarrage et retour au menu

- Écran de pause pendant le jeu

Système de score :

- Calcul des points selon le niveau et lignes complétées à la fois
- Augmentation du niveau après chaque 10 lignes complétées
- Affichage en temps réel du score, niveau et lignes

2.2 Jouabilité et Contrôles

Le jeu propose deux modes de contrôle selon le type de jeu :

2.2.1 Mode Un Joueur

Fonction	Touche	Description
Déplacement gauche	←	Déplace la pièce actuelle d'une case vers la gauche
Déplacement droit	→	Déplace la pièce actuelle d'une case vers la droite
Rotation	↑	Effectue une rotation de 90° dans le sens horaire
Descente lente	↓	Accélère la descente de la pièce (<i>soft drop</i>)
Chute instantanée	Space	Fait tomber la pièce instantanément à sa position finale (<i>hard drop</i>)
Hold	C	Conserve la pièce actuelle en réserve et sort la pièce précédemment conservée
Pause	Esc	Met le jeu en pause/affiche le menu
Redémarrage	R	Redémarre la partie en cours

2.2.2 Mode Deux Joueurs Local

Fonction	Joueur 1	Joueur 2
Gauche	←	A
Droite	→	D
Rotation	↑	W
Descente lente	↓	S
Chute instantanée	RShift	LShift
Hold	RControl	LControl

3 Architecture & Implementation

Le système suit une architecture Modèle-Vue-Contrôleur (MVC) rigoureusement structurée en trois composants principaux séparés, permettant une maintenance et une extensibilité optimales.

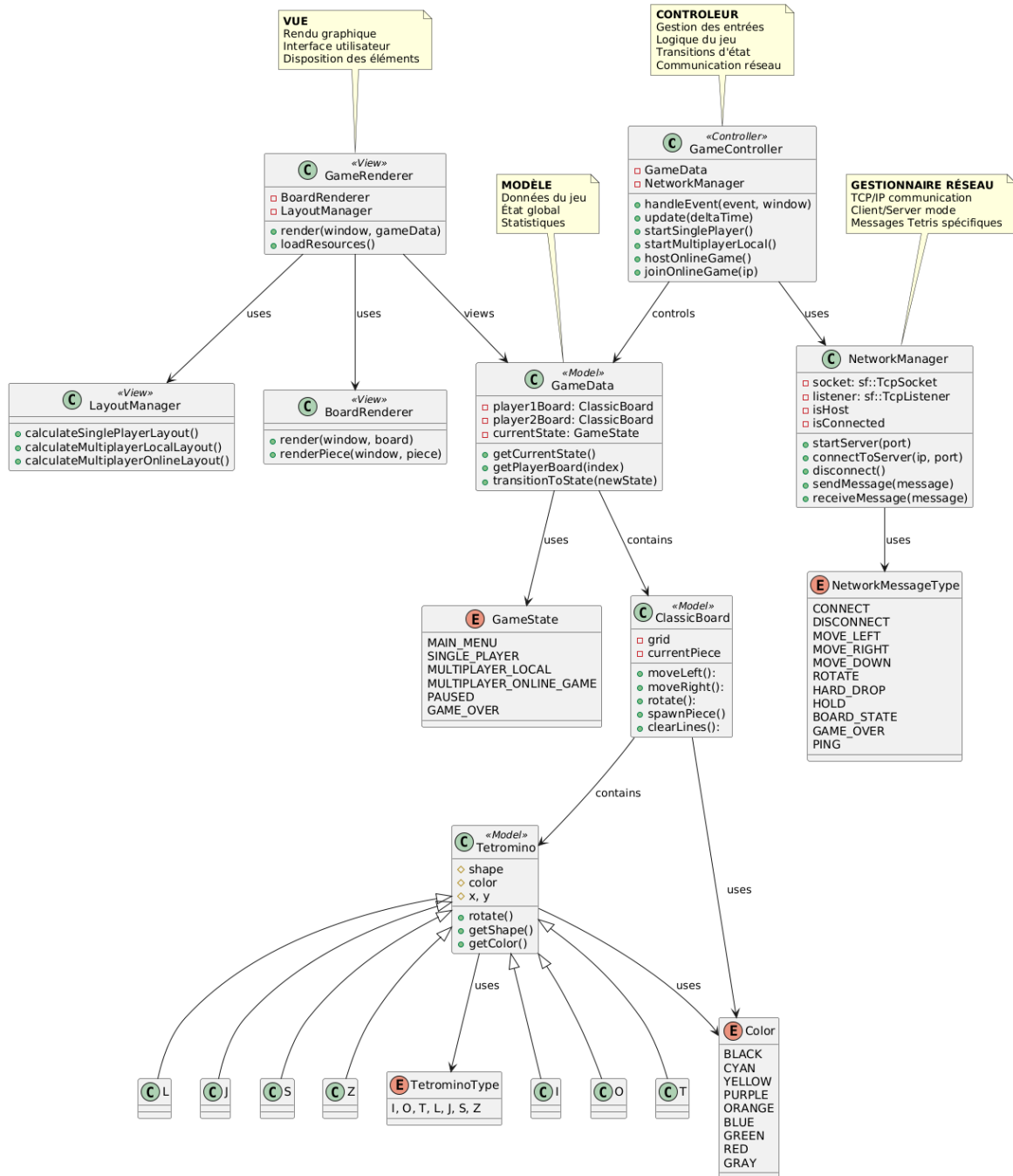


FIGURE 1 – Diagramme de classes du système Tetris

3.1 Machine à États Finis

Le cœur de l'architecture est une machine à 7 états implémentée dans le contrôleur `GameController`. Cette approche permet de gérer proprement les différents modes de jeu et les transitions entre eux, chaque état ayant ses propres gestionnaires d'événements, de mise à jour et de rendu.

États Principaux :

1. **MAIN_MENU** : État initial, présente les options de jeu via `GameRenderer`
2. **SINGLE_PLAYER** : Mode un joueur classique géré par `ClassicBoard`
3. **MULTIPLAYER_LOCAL** : Deux joueurs sur le même ordinateur avec synchronisation locale
4. **MULTIPLAYER_ONLINE_LOBBY** : Salle d'attente pour le jeu en ligne gérée par `NetworkManager`
5. **MULTIPLAYER_ONLINE_GAME** : Jeu en réseau avec un adversaire distant
6. **GAME_OVER** : Écran de fin de partie avec score final rendu par `GameRenderer`
7. **PAUSED** : Écran de pause pendant le jeu

Transitions d'État : Le système gère les transitions via la méthode `handleEvent()` du `GameController` qui, selon l'état courant du `GameData`, appelle le gestionnaire approprié. Par exemple :

- De **MAIN_MENU** à **SINGLE_PLAYER** : Lorsque l'utilisateur sélectionne "Single Player"
- De **SINGLE_PLAYER** à **GAME_OVER** : Lorsque le plateau est plein

3.2 Composants MVC

3.2.1 Modèle (`GameData`, `ClassicBoard`, `Tetromino`)

Structure de Données Centralisée

Le modèle encapsule toute la logique métier et les données du jeu :

- **GameData** : Gère l'état global du jeu, les statistiques des joueurs et les configurations
- **ClassicBoard** : Implémente la logique du jeu Tetris avec gestion des collisions et du score
- **Tetromino** : Hiérarchie des pièces avec système de rotation et positionnement

Hiérarchie des Pièces (`Tetromino`)

Le système de pièces utilise une hiérarchie de classes où `Tetromino` est la classe abstraite de base. Chaque type de pièce (I, O, T, L, J, S, Z) est implémenté comme une sous-classe concrète. Cette structure permet :

- **Encapsulation** : Chaque pièce gère sa propre forme et couleur
- **Polymorphisme** : Toutes les pièces partagent la même interface via la classe de base

Factory Method Pattern

La méthode `Tetromino::create()` implémente le pattern Factory Method pour la création d'instances de pièces :

- **Abstraction** : Le code client n'a pas besoin de connaître les classes concrètes
- **Centralisation** : Toute la logique de création est dans une seule méthode

Fonctionnement : La méthode prend un `TetrominoType` en paramètre et retourne un `unique_ptr` vers l'instance appropriée. Elle utilise un switch sur l'énumération pour sélectionner la classe concrète à instancier.

Prototype Pattern

La méthode `Tetromino::clone()` implémente le pattern Prototype pour créer des copies profondes des pièces. Ce pattern est essentiel pour la création de la *ghost piece* (une copie de la pièce actuelle pour la visualisation de la position finale).

Implémentation : La méthode clone crée une nouvelle instance via la factory, puis copie tous les attributs (forme, couleur, position) de l'instance originale.

Système de Collision

La détection de collision est centralisée dans la méthode `isValidPosition()` de `ClassicBoard`. Cette méthode vérifie si la pièce sort des limites horizontales ou descend en dessous du plateau - ou des blocs déjà placés.

Système de Score

Le calcul du score suit les règles classiques du Tetris prenant en compte le niveau et nombre de lignes complétées à la fois, ayant une augmentation de niveau tous les 10 lignes complétées :

Lignes Complétées	Points de Base	Formule
1 ligne	40	$40 \times (\text{niveau} + 1)$
2 lignes	100	$100 \times (\text{niveau} + 1)$
3 lignes	300	$300 \times (\text{niveau} + 1)$
4 lignes	1200	$1200 \times (\text{niveau} + 1)$

TABLE 1 – Système de calcul des points

Système de Contrôle et d'Assistance

Quelques mécanismes ont été implémentés pour améliorer la fluidité du jeu et l'expérience d'utilisateur :

- **Hold** : Permet au joueur de conserver une pièce pour une utilisation ultérieure
- **Wall kick** : Permet les rotations même lorsque la pièce est contre un mur ou d'autres blocs. L'implémentation teste une séquence de décalages et le système s'arrête au premier décalage valide, offrant un comportement prédictible
- **Ghost piece** : Montre où la pièce actuelle atterrirait si elle tombait immédiatement. La pièce est clonée avec transparence via `Tetromino::clone()` et `BoardRenderer` et est fait descendre jusqu'à collision

3.2.2 Vue (GameRenderer, BoardRenderer, LayoutManager)

Le système de rendu n'accède au modèle `GameData` qu'en lecture seule, préservant la séparation MVC. Il est divisé en trois composants spécialisés :

- `GameRenderer` : Rendu principal selon l'état du jeu
- `BoardRenderer` : Rendu spécifique des plateaux Tetris, avec mappage des couleurs pour chaque type de pièce et application d'une transparence pour la *ghost piece*
- `LayoutManager` : Gestion du positionnement des éléments UI, calcule dynamique des positions selon la taille de la fenêtre et mode de jeu

3.2.3 Contrôleur (GameController)

Gestion des Entrées et Mise à Jour

Le contrôleur traite tous les événements d'entrée via `handleEvent()`. Chaque état a son propre gestionnaire d'événements, avec touches différents selon le mode. En plus, la méthode `update()` exécute la logique du jeu, par la gestion des timers de chute, des appels de mise à jour spécifiques pour chaque mode de jeu (comme `updateSinglePlayer()` et `updateMultiplayerLocal()`), et la synchronisation réseau pour l'envoi/réception d'états dans le mode en ligne.

Gestion de la Vitesse

La vitesse de chute des pièces augmente avec le niveau selon la formule suivante :

$$\text{Vitesse} = \max(0.05, 1.0 - (\text{Niveau} \times 0.1)) \text{ secondes/descente}$$

Niveau 1 : 0.9 seconde par descente

Niveau 5 : 0.5 seconde par descente

Niveau 10 : 0.05 seconde par descente (vitesse maximale)

3.3 Réseau (NetworkManager)

Architecture Client-Serveur

Le système réseau utilise une architecture client-serveur avec TCP/IP implémentée dans la classe `NetworkManager` :

- **Hôte** : Écoute sur le port 54000 via `startServer()`, accepte une connexion via `waitForClient()`
- **Client** : Se connecte à l'adresse IP de l'hôte via `connectToServer()`

Protocole de Communication

Le protocole utilise des messages typés définis par l'énumération `NetworkMessageType` :

Type	Code	Description
CONNECT	0	Établissement de connexion initial
DISCONNECT	1	Fermeture de connexion
MOVE_LEFT	2	Déplacement de pièce vers la gauche
MOVE_RIGHT	3	Déplacement de pièce vers la droite
MOVE_DOWN	4	Descente rapide de pièce
ROTATE	5	Rotation de pièce
HARD_DROP	6	Chute immédiate de pièce
HOLD	7	Mise en réserve de pièce
BOARD_STATE	8	Mise à jour des statistiques du plateau
GAME_OVER	9	Notification de fin de partie
PING	10	Message de maintien de connexion

TABLE 2 – Types de messages réseau

Structure des Messages

Chaque message réseau suit la structure `NetworkMessage` et est communiqué à l'aide de la méthode *Peer to Peer* :

- **type** : Type de message (`NetworkMessageType`)
- **playerID** : Identifiant du joueur émetteur
- **data1** : Donnée supplémentaire (score, niveau, etc.)
- **data2** : Donnée supplémentaire (lignes, etc.)

Exemple : Un message `BOARD_STATE` contient dans **data1** le score et dans **data2** le nombre de lignes complétées.

Gestion de la Connexion

La connexion réseau est gérée de manière non-bloquante :

- **Initialisation** : Mode bloquant pour la connexion initiale
- **Jeu** : Mode non-bloquant pendant le jeu avec `sendMessage()` et `receiveMessage()`

3.4 Boucle de Jeu Principale

La boucle de jeu dans `main.cpp` suit le pattern classique :

1. **Initialisation** : Création des composants MVC
2. **Événements** : `controller.handleEvent()`
3. **Mise à jour** : `controller.update()`
4. **Ajustement vue** : `setupViewForState()`
5. **Rendu** : `renderer.render()`

3.5 Utils

Organisation des Constantes

Les fichiers utils centralisent les définitions partagées :

- `color.hpp` : Définitions RGB pour chaque type de pièce
- `game_state.hpp` : Énumérations des états et options de menu

4 Conclusion

Le projet Tetris a été développé avec une architecture Modèle-Vue-Contrôleur qui sépare clairement les responsabilités entre le contrôleur `GameController`, les modèles `GameData` et `ClassicBoard`, et les vues `GameRenderer` et `BoardRenderer`. Cette approche, combinée à une machine à états finis pour gérer les différents modes de jeu, a permis de créer un système modulaire, maintenable et extensible tout en implémentant les règles classiques du Tetris avec trois modes de jeu distincts.

L'architecture MVC adoptée a démontré des avantages significatifs en termes de séparation des préoccupations, testabilité et extensibilité. Bien que des améliorations soient possibles, notamment pour la synchronisation réseau et l'enrichissement visuel, la base technique établie constitue une solide fondation pour des développements futurs, illustrant l'importance des choix architecturaux appropriés pour le développement d'un projet de jeu vidéo.